JOMHACK
EMPOWERING INNOVATION

```
> bootcamp.learn()
```

JOMHACK
EMPOWERING INNOVATION

JomCode Coding Bootcamp

CHAPTER 09

Errors, debugging and basic testing

`learn / type / run / explain`

Plain meaning and technical explanation

PLAIN MEANING

Failures provide clues. Checks help prove that a fix works.

TECHNICAL EXPLANATION

Exceptions interrupt normal execution. try/except handles expected failures. Tracebacks identify the call path; assertions check programmer expectations.

REAL-WORLD USE

Find bad input conversions and verify validation boundaries.

QUICK TIP

Read the final exception message, then inspect the referenced line and its inputs.

INSTALL / RUN

No packages. Windows: py workbook.py. macOS: python3 workbook.py. VS Code: breakpoint, then Run and Debug.

Worked example 1: Handle a known conversion error

workbook.py

```
01    def parse_count(text):
02        try:
03            return int(text)
04        except ValueError:
05            return None
06
07    print(parse_count("12"))
08    print(parse_count("twelve"))
```

PRO TIP Avoid a bare except that hides unrelated bugs.

Run and explain example 1

EXPECTED RESULT

12
None

WHY IT BEHAVES THIS WAY

Only ValueError is handled. Unexpected failures remain visible.

Worked example 2: Raise a useful error

workbook.py

```
01    def clean_title(text):
02        title = text.strip()
03        if not title:
04            raise ValueError("Title is required")
05        return title
06
07    try:
08        clean_title("   ")
09    except ValueError as error:
10        print(error)
```

PRO TIP Use an exception message that identifies the violated rule.

Run and explain example 2

EXPECTED RESULT

Title is required

WHY IT BEHAVES THIS WAY

Raising an exception prevents invalid data from continuing through the program.

Worked example 3: Boundary assertions

workbook.py

```
def allowed_size(size):
    return 0 < size <= 5 * 1024 * 1024

assert not allowed_size(0)
assert allowed_size(5 * 1024 * 1024)
assert not allowed_size(5 * 1024 * 1024 + 1)
print("3 checks passed")
```

PRO TIP Assertions can be disabled; use explicit validation for user input.

Run and explain example 3

EXPECTED RESULT

3 checks passed

WHY IT BEHAVES THIS WAY

Boundary checks catch off-by-one mistakes at the upload limit.

Worked example 4: Inspect intermediate values

workbook.py

```
raw = "  Hello  "
cleaned = raw.strip()
print(repr(raw), len(raw))
print(repr(cleaned), len(cleaned))
assert len(cleaned) == 5
```

PRO TIP Set a breakpoint before the assertion and inspect both variables.

Run and explain example 4

EXPECTED RESULT

' Hello ' 9
'Hello' 5

WHY IT BEHAVES THIS WAY

repr() reveals whitespace that ordinary output can hide. A debugger can inspect these values without extra prints.

Exercise 1: Nonnegative counts

YOUR TASK

Convert numeric text to a nonnegative integer or return None.

CHECK THESE CASES

"0" gives 0. "-1" and "abc" give None.

HINT AFTER A TRY

Catch ValueError, then check the numeric range.

Exercise 2: Regression checks

YOUR TASK

Test title cleaning for a normal, blank and overlong title.

CHECK THESE CASES

Normal title succeeds. Blank and 161-character titles raise ValueError.

HINT AFTER A TRY

Use a loop and explicitly fail if an invalid case succeeds.

Checkpoint before moving on

01 Run and explain all four worked examples. Match their actual results to the expected behavior.

02 Attempt both exercises before reading the solutions. Verify every acceptance case.

03 Explain how this chapter supports Postlight: Find bad input conversions and verify validation boundaries.

Official reference